ARTIFICIAL INTELLIGENCE ESSENTIALS

PROJECT REPORT

DRIFTX: A SELF-HEALING AI SYSTEM

Submitted by

Krishika Malhotra

Registration No.: 12413944

Section: 424LP

Roll No.: 61

Under the Guidance of

Dr. Avinash Kaur

Discipline of CSE/IT

Lovely School of Computer Science

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that **Krishika Malhotra**, bearing Registration No. **12413944**, has completed the project titled **"DriftX: A Self-Healing AI System"** under my guidance and supervision. To the best of my knowledge, the present work is the result of her original development, effort, and study.

Signature and Name of the Supervisor: _____
Designation of the Supervisor: _____
School of Computer Science
Lovely Professional University
Phagwara, Punjab

Date: _____

DECLARATION

I, **Krishika Malhotra**, student of Bachelor of Technology under the CSE/IT discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own work and is genuine.

Date: April 21, 2026
Name of the Student: **Krishika Malhotra**
Registration No.: **12413944**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor for continuous guidance, encouragement, and support throughout the completion of this project. Their suggestions helped shape both the technical implementation and the presentation of the work.

I also thank Lovely Professional University for providing a learning environment that encouraged experimentation and project-based application of artificial intelligence concepts. In addition, I acknowledge the role of online technical documentation, developer communities, and academic resources that supported the understanding of browser extension architecture, DOM manipulation, and AI reliability concepts.

Finally, I extend my thanks to my peers and well-wishers for their valuable feedback and motivation during the development of DriftX.

TABLE OF CONTENTS

1. Introduction
2. Background and Theoretical Concepts
3. Dataset Preprocessing
4. Objectives
5. System Design and Architecture
6. Dataset and Preprocessing
7. Model Development
8. Drift Simulation, Detection and Recovery
9. Conversational AI Component
10. Results and Evaluation
11. Limitations
12. Future Scope
13. Conclusion
14. References

1. INTRODUCTION

Artificial Intelligence systems have become increasingly capable in tasks such as conversation, prediction, recommendation, and automation. However, capability alone does not guarantee reliability. In many practical situations, AI systems may produce unstable, inconsistent, or incorrect outputs when the surrounding context changes. This problem becomes especially serious when users begin to trust AI responses without mechanisms for monitoring or recovery.

One important challenge in AI reliability is **drift**. Drift refers to the gradual or sudden deviation of system behavior from an expected or stable pattern. In conversational AI systems, this can appear as incorrect facts, unstable responses, irrelevant answers, or confidently wrong outputs. While modern AI systems are highly intelligent, they often do not include built-in self-recovery mechanisms visible to the end user.

To address this conceptual gap, this project introduces **DriftX**, a self-healing AI system designed to demonstrate the ideas of drift, instability, and recovery. In the present implementation, DriftX is demonstrated through a **Chrome extension interface** that operates on top of chatbot webpages such as ChatGPT. The extension captures the latest visible chatbot response, stores it as a stable state, simulates a drifted output, and then restores the original output through a healing operation.

Unlike traditional chatbot projects that focus only on producing answers, DriftX emphasizes a broader systems perspective: **AI reliability, failure visibility, and recovery behavior**. The project is therefore not just about generating responses, but about illustrating how future AI systems can move from intelligence toward reliability.

2. BACKGROUND AND THEORETICAL CONCEPTS

2.1 Artificial Intelligence Reliability

Artificial Intelligence reliability refers to the ability of an AI-driven system to behave consistently, predictably, and safely over time. A reliable AI system should not only provide useful outputs under normal conditions, but should also respond gracefully when its environment, inputs, or context change.

2.2 Drift in AI Systems

The concept of drift is commonly discussed in machine learning and production AI systems. Broadly, drift refers to a mismatch between what a system expects and what it actually receives or processes over time. The major categories include:

- **Data Drift**: A shift in the distribution of input data.
- **Concept Drift**: A change in the relationship between inputs and expected outputs.
- **Context Drift**: A shift in the meaning, relevance, or interpretation of information, especially common in conversational systems.

In this project, the focus is primarily on **context drift** as it appears in chatbot responses.

2.3 Effects of Drift

Drift can lead to a range of negative outcomes:

- Degraded response quality
- Reduced consistency
- Incorrect or misleading outputs
- Loss of user trust

In conversational AI, drift is particularly difficult because the system may remain fluent and convincing even when the content has become unreliable.

2.4 Self-Healing Systems

A self-healing system is one that can recover from an error state and return to a more stable or trusted state. In large-scale computing systems, this may involve restarting services, restoring configurations, or routing around failures. In the context of DriftX, self-healing is demonstrated by restoring the original stable chatbot response after a simulated drift event.

2.5 DOM-Based Demonstration Layer

The practical implementation of DriftX relies on the **Document Object Model (DOM)** of the webpage. The DOM is the browser's internal representation of webpage content as a structured tree of elements. By interacting with the DOM, JavaScript can locate chatbot responses, change visible text, inject interface elements, and restore prior content. This makes the browser an effective environment for demonstrating drift without modifying the actual AI model.

3. DATASET PREPROCESSING

In a conventional machine learning project, dataset preprocessing refers to cleaning, transforming, and structuring raw data before training a model. DriftX differs from such systems because the present implementation is **not based on a static training dataset**. Instead, it operates on **live webpage content** generated by an external chatbot system.

For this reason, preprocessing in DriftX takes the form of **runtime response preparation** rather than offline dataset engineering. The system processes:

- The latest visible user prompt
- The latest visible assistant response
- The HTML structure of the chatbot response block

Before any drift or healing operation is performed, the extension prepares the response through:

- DOM selection of the relevant assistant message
- Visibility filtering to avoid hidden or irrelevant elements
- Text normalization to remove excessive spacing
- Storage of the original HTML structure for exact restoration

This preprocessing step is essential because the extension must operate reliably on dynamic webpage content rather than on fixed tabular records.

4. OBJECTIVES

The following objectives guided the development of DriftX:

1. To study the problem of drift in AI systems, especially in conversational environments.

2. To design a system that can clearly demonstrate drift without modifying the underlying AI model.

3. To build a browser-based interface capable of capturing chatbot responses in real time.

4. To simulate different forms of visible response degradation, including textual, numeric, and code-based drift.

5. To implement a self-healing mechanism that restores the original trusted response.

6. To create a system that is safe, presentable, and suitable for academic demonstration.

7. To bridge theoretical AI reliability concepts with an interactive, user-facing prototype.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 Architectural Overview

DriftX follows a lightweight, modular architecture based on a **browser extension control layer** and a **page interaction layer**. The system is not implemented as a standalone AI model. Instead, it behaves as a reliability-oriented overlay on top of an existing chatbot webpage.

At a high level, the architecture consists of:

- **Manifest Layer**
- **Popup Interface Layer**
- **Messaging Layer**
- **Content Script / DOM Manipulation Layer**
- **Drift and Healing Logic Layer**

5.2 Manifest Layer

The `manifest.json` file defines the extension's identity, permissions, popup file, and the websites on which the content script should run. This layer is responsible for allowing the system to operate on ChatGPT-compatible pages while remaining restricted to the required domains.

5.3 Popup Interface Layer

The popup layer consists of `popup.html`, `popup.js`, and `style.css`. This layer acts as the user control interface and provides:

- An **Open ChatGPT** action
- A **Drift** button
- A **Heal** button
- A **status display** showing whether the system is ready, drifted, healed, or unsupported

5.4 Messaging Layer

The popup cannot directly manipulate the webpage. Therefore, the system uses Chrome's message-passing mechanism through `chrome.tabs.sendMessage()`. The popup sends action commands such as:

- 'status'
- 'drift'
- 'heal'

These commands are then received and processed by the content script.

5.5 Content Script Layer

The `content.js` file runs inside the ChatGPT webpage. This is the operational core of DriftX. It performs the following tasks:

- Locates the latest visible assistant response
- Locates the latest visible user prompt
- Stores the original response text and HTML
- Determines the response type
- Generates a simulated drifted version
- Replaces the visible answer in the DOM
- Restores the original answer during healing

5.6 Working Principle

The complete system flow is as follows:

1. The user opens a chatbot page such as ChatGPT.
2. The content script becomes active on that page.
3. The user clicks the DriftX extension popup.
4. The popup sends a command to the content script.
5. The content script captures the latest response and stores it as the baseline.
6. A drifted response is generated and injected into the DOM.
7. On heal, the original stored response is restored.

This architecture ensures that DriftX remains **non-invasive**, **demo-safe**, and **easy to understand**.

6. DATASET AND PREPROCESSING

Although DriftX does not rely on a traditional ML dataset in its current extension-based implementation, it still works with a meaningful runtime data structure. The “working dataset” of the system consists of:

- User prompts visible on the webpage
- Assistant responses visible on the webpage
- Response formatting in HTML form

6.1 Runtime Data Description

Each interaction cycle uses:

- The most recent user prompt as contextual input
- The most recent assistant answer as the response to be monitored
- The DOM element containing the answer as the editable interface block

6.2 Runtime Preprocessing Pipeline

The preprocessing pipeline includes:

6.2.1 DOM Selection

The system queries the webpage for assistant response elements using selectors aligned with the ChatGPT interface structure.

6.2.2 Visibility Filtering

Only visible elements are considered valid. This reduces the chance of selecting hidden UI components or irrelevant cached elements.

6.2.3 Whitespace Normalization

Response text is normalized before drift generation so that transformations can operate consistently.

6.2.4 Prompt-Response Pairing

The system stores the latest user prompt alongside the captured response so that drift generation can be made context-aware.

6.2.5 Original HTML Storage

The response is stored not only as plain text but also as full HTML. This preserves formatting and allows the system to heal the response accurately.

7. MODEL DEVELOPMENT

The current implementation of DriftX does **not** use a trained machine learning model within the extension itself. Instead, it uses a **rule-based heuristic engine** written in JavaScript.

7.1 Heuristic Logic vs Machine Learning

In machine learning, a system learns patterns from data through training. In DriftX, no such training is performed. The behavior is manually defined through conditional logic. For example:

- Numeric outputs are transformed into incorrect numeric outputs
- Code responses are transformed into incorrect code snippets
- Textual answers are replaced with wrong but context-relevant textual answers

This approach was chosen because the purpose of the project is to **demonstrate reliability concepts clearly and safely**, rather than to build a full internal drift-detection model.

7.2 Response Classification Logic

The system uses heuristic classification for:

- **Topic detection**: politics, math, code, geography, science, or general
- **Answer-shape detection**: numeric, code, or text

These classifications are based on:

- Keywords in the user prompt
- Patterns in the assistant response
- HTML tags such as `<pre>` and `<code>` for code detection

7.3 Why a Heuristic Engine Was Used

The heuristic approach offers several advantages in the current prototype:

- Simplicity of implementation
- Immediate interpretability
- Strong suitability for live demonstration
- No requirement for training data
- Safe interaction with third-party chatbot webpages

Thus, while DriftX is conceptually an AI reliability system, the present browser-extension implementation should be understood as a **heuristic simulation prototype**, not as a trained standalone model.

8. DRIFT SIMULATION, DETECTION AND RECOVERY

8.1 Drift Simulation Methodology

DriftX simulates drift by intentionally replacing the visible chatbot answer with a wrong answer generated through rule-based logic. The drift is designed to remain recognizable, topic-sensitive, and visibly different from the original answer.

Examples include:

- Changing factual answers to incorrect names
- Replacing numeric outputs with different numeric results
- Replacing code outputs with incorrect or misleading code snippets
- Replacing short text answers such as colours with clearly wrong alternatives

8.2 Drift Detection Approach

The current prototype does not implement statistical drift detection algorithms such as:

- KL Divergence
- Population Stability Index (PSI)
- KS-Test
- ADWIN

Instead, drift is operationalized as a controlled simulation event. The system knows drift has occurred because it is triggered explicitly by the user through the Drift button. In this sense, the extension demonstrates **drift injection and recovery**, not automated real-time drift discovery.

8.3 State Preservation

Before drift is applied, the original response is captured and stored in memory:

- Original plain text
- Original HTML
- Original prompt
- Current response element reference

This stored state acts as the system's baseline for healing.

8.4 Self-Healing Mechanism

When healing is triggered, the stored original HTML is restored to the same DOM element. This returns the page to its prior visible state, including formatting. The mechanism is deterministic and immediate.

This implementation models the idea of self-healing as **state restoration** rather than model retraining. It is therefore best understood as a UI-level analogue of recovery behavior in more advanced AI systems.

8.5 Reliability Interpretation

The prototype demonstrates an important principle: AI reliability is not only about generating answers, but also about preserving, monitoring, and recovering trusted states. DriftX makes this principle visible through a direct before-drift-after flow.

9. CONVERSATIONAL AI COMPONENT

DriftX is designed to operate on top of conversational AI systems, particularly interfaces such as ChatGPT. The conversational AI component in this project should be understood as **external** to DriftX itself.

9.1 External Chatbot Dependency

The original response is generated by the underlying chatbot platform. DriftX does not generate the initial answer. Instead, it observes and manipulates the rendered webpage response.

9.2 Browser-Level Integration

The connection to conversational AI is achieved through:

- A Chrome content script
- DOM inspection
- Page-level message handling

This means the system is connected to the chatbot through the browser interface, not through internal model access or a direct backend API integration in the current prototype.

9.3 Context-Aware Drift in Chat Mode

DriftX uses the latest user prompt to shape the type of drift applied. For example:

- A colour question may drift to “green” or “pink”
- A person-identification question may drift to an incorrect name
- A coding prompt may drift into a wrong code block
- A numeric question may drift into a different numerical answer

This makes the conversation-level demonstration more realistic and more useful for explaining context drift.

9.4 Educational Significance

The conversational component is especially valuable because it illustrates that drift is not limited to structured tabular prediction systems. It can also affect language-based systems where the error is semantic and contextual rather than purely statistical.

10. RESULTS AND EVALUATION

The evaluation of DriftX in its present form is primarily **functional and demonstrative**, rather than benchmark-driven.

10.1 Functional Outcomes

The system successfully demonstrated the following behaviors:

- Identification of the latest visible chatbot response
- Storage of the original response as a restorable baseline
- Drift simulation across numeric, textual, and code-like outputs
- Immediate restoration of the original response through healing
- Status feedback through the popup interface

10.2 User-Facing Demonstration Value

The extension effectively communicates the core reliability narrative:

- Stable response
- Drifted response
- Healed response

This visual sequence is highly useful in presentations because it transforms an abstract theoretical concept into a direct observable event.

10.3 Technical Evaluation

The current system performed well in the following practical areas:

- Instant DOM-level updates without reloading the page
- Separation of control logic and webpage logic
- Context-sensitive drift generation
- Preservation of formatting during healing

10.4 Qualitative Interpretation

From a systems perspective, the main success of DriftX lies not in predictive accuracy, but in **clarity of demonstration**. The project makes AI drift visible, understandable, and recoverable through a simple interactive mechanism.

11. LIMITATIONS

The present implementation of DriftX has several limitations that are important for proper academic evaluation:

1. **No Real Statistical Drift Detection**
   The system does not currently use KL Divergence, PSI, KS-Test, or other statistical methods for detecting drift in live data streams.

2. **No Ground-Truth Validation**
   The stored original response is treated as the stable baseline, but the system does not independently verify whether that original response is actually correct.

3. **No Internal Model Access**
   DriftX does not connect to the internal reasoning process of ChatGPT or another AI model. It only manipulates the rendered webpage output.

4. **No Learning-Based Recovery**
   Healing is implemented through restoration of a previously stored state, not through retraining, feedback learning, or model adaptation.

5. **Platform Dependence**
   The DOM selectors are designed for ChatGPT-like interfaces. If the structure of the target website changes significantly, the extension may require adjustment.

6. **Prototype Scope**
   The system is best understood as a proof-of-concept for AI reliability demonstration rather than a production-grade monitoring framework.

12. FUTURE SCOPE

DriftX can be significantly extended in future work. Important directions include:

1. **Real Drift Detection Algorithms**
   Integrating methods such as KL Divergence, PSI, KS-Test, or Page-Hinkley for automatic drift detection.

2. **Confidence Scoring Framework**
   Adding a confidence-aware baseline selection mechanism so that only trusted responses are stored as healable states.

3. **Feedback Loops and Validation Pipelines**
   Allowing the system to learn from corrections, user feedback, or verification services.

4. **Backend AI Reliability Engine**
   Extending DriftX from a browser demo layer into a backend middleware system that sits between users and AI models.

5. **Multi-Platform Support**
   Supporting additional chatbot interfaces and browser environments.

6. **Agentic Reliability Layer**
   A future version could include autonomous monitoring, anomaly checking, and recovery decision-making, moving the system closer to agentic AI behavior.

7. **Dashboard and Analytics**

A visual dashboard for tracking drift frequency, recovery cycles, and response stability over time.

13. CONCLUSION

This report presented **DriftX**, a self-healing AI system concept demonstrated through a Chrome extension interface. The project addresses the growing need for reliability-oriented thinking in artificial intelligence by making the phenomenon of drift visible, understandable, and recoverable in a live conversational environment.

The system shows that an AI application may appear intelligent while still remaining vulnerable to instability and context drift. By capturing the latest chatbot response, simulating a controlled drift event, and restoring the original stable response through a healing mechanism, DriftX provides a clear and engaging demonstration of reliability engineering concepts in AI.

Although the current implementation does not use trained internal models, statistical drift-detection algorithms, or autonomous feedback loops, it offers strong educational value. It bridges theory and practice by showing how concepts such as drift, trust, fallback, restoration, and system stability can be represented through an interactive software system.

In this sense, DriftX is not merely a browser utility. It is a prototype of a broader idea: that the future of artificial intelligence must include not only intelligence, but also **monitoring, resilience, and self-healing behavior**.

14. REFERENCES

1. T. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.

2. J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.

3. H. Wang, W. Fan, P. S. Yu, and J. Han, "Mining concept-drifting data streams using ensemble classifiers," in *Proceedings of ACM SIGKDD*, 2003.

4. M. Baena-García et al., "Early drift detection method," in *Proceedings of ECML Workshop on Knowledge Discovery from Data Streams*, 2006.

5. J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

6. OpenAI, "OpenAI API Documentation." [Online]. Available: https://platform.openai.com/docs

7. Google Chrome Developers, "Chrome Extensions Documentation." [Online]. Available: https://developer.chrome.com/docs/extensions

8. Mozilla Developer Network, "Introduction to the DOM." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model